

BR-MPPI: Barrier-Rate Guided MPPI for Enforcing Multiple Inequality Constraints with Learned Signed Distance Fields

Hardik Parwana^{1,*}, Taekyung Kim^{1,*}, Kehan Long², Bardh Hoxha³, Hideki Okamoto³, Georgios Fainekos³, and Dimitra Panagou¹

Abstract—Model Predictive Path Integral (MPPI) control provides a sampling-based framework for optimal control, while Control Barrier Functions (CBFs) provide a principled means of enforcing safety constraints. We introduce BR-MPPI, which integrates *CBF-like* conditions into MPPI’s control sampling procedure. CBFs impose inequality constraints that bound the rate of change of barrier functions using a class- $\mathcal{K}$ function of the barrier value. We instead impose the CBF condition as an *equality constraint* using a parametric linear class- $\mathcal{K}$ function and augment the system state with its parameter. The parameter’s time derivative serves as an additional control input optimized by MPPI. We further design a cost function that promotes parameter values consistent with Nagumo’s condition at the safe-set boundary, thereby encouraging safety. The resulting multiple state- and control-dependent equality constraints pose a challenge for random control sampling. We address this through state transformations and control projections inspired by manifold path planning that map sampled controls onto the constraint manifold. We also incorporate learned signed distance fields to represent robot geometry and reduce computation time. Simulations demonstrate improved sample efficiency over vanilla MPPI and higher navigation success rates across five robot models compared with safety-oriented MPPI variants. Hardware experiments on a quadrotor further demonstrate the method’s ability to navigate constrained environments near safe-set boundaries. [Project Page]¹ [Code] [Video]

I. INTRODUCTION

Autonomous systems increasingly incorporate learning-based techniques into real-time operation, from active learning for estimating unknown quantities to offline-learned dynamics models and signed distance fields. Deploying these systems in constrained environments requires real-time kinodynamic planning that accounts for robot geometry and permits operation near safe-set boundaries. Model Predictive Path Integral (MPPI) control [1] and Control Barrier Function (CBF)-based quadratic programs (QPs) [2] offer complementary approaches to this problem. MPPI optimizes trajectories over a finite horizon but does not inherently guarantee inequality constraint satisfaction, whereas standard

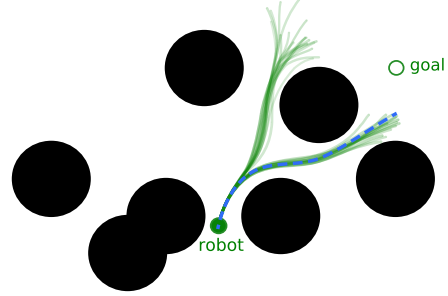


Fig. 1. BR-MPPI navigation among obstacles (black). Green curves show sampled paths, and the blue curve shows the path generated by the updated control sequence. Planning in the augmented state space produces distinct groups of sampled paths on different sides of obstacles in the robot’s original state space.

CBF-QP controllers enforce instantaneous safety conditions but can be myopic and yield suboptimal trajectories.

Vanilla MPPI [1] solves unconstrained optimal control problems by sampling control sequences, evaluating the resulting state trajectories, and updating the nominal sequence through a cost-weighted average of the sampled controls. Cost penalties provide a soft treatment of inequality constraints, leaving MPPI susceptible to violations, particularly with few samples [3]. Penalty weights may also require extensive tuning to navigate close to constraint boundaries without violating them. Among the extensions developed to improve MPPI [4]–[7], we focus on CBF-based approaches to constraint satisfaction.

A straightforward approach is to filter MPPI commands through a CBF-QP [3]. Alternatively, the same work constructs trust regions by optimizing the mean and covariance of the control sampling distribution subject to CBF conditions. This requires a semidefinite program (SDP) at each time step of every sampled trajectory, with fewer samples used to mitigate the computational burden. Shield-MPPI [8] for deterministic systems and the method in [9] for stochastic systems penalize CBF inequality violations and subsequently apply gradient-based nonlinear optimization to enforce the constraints, subject to convergence. GS-MPPI [10] instead applies a closed-form CBF-based safety correction.

SC-MPPI [11] incorporates barrier-state-based safe feedback into sampling. The related formulation in [12] augments the robot state with barrier values that diverge at constraint boundaries. These added states are functions of the robot state: their evolution is determined by the robot trajectory,

*These authors contributed equally to this work

¹Robotics Department, University of Michigan, Ann Arbor, MI 48109, USA {hardiksp, taekyung, dpanagou}@umich.edu

²Contextual Robotics Institute, University of California San Diego, La Jolla, CA 92093, USA. k3long@ucsd.edu.

³Toyota Motor North America, Research & Development, Ann Arbor, MI 48105, USA. <first_name.last_name>@toyota.com

This work was primarily conducted during the summer internships of Hardik Parwana, Taekyung Kim, and Kehan Long at Toyota Motor North America R & D.

¹Project page: <https://www.taekyung.me/br-mppi>

rather than governed by additional control inputs.

We introduce Barrier-Rate Guided MPPI (BR-MPPI), which incorporates *CBF-like* conditions directly into sampling without the inequality-constrained optimization used in [3]. CBFs impose inequality constraints that bound the rate of change of barrier functions using a class- $\mathcal{K}$ function of the barrier value. For each constraint, we instead impose the CBF condition as an *equality constraint* using a parametric linear class- $\mathcal{K}$ function and augment the system state with its parameter rather than the barrier value. The parameter's time derivative serves as an additional control input optimized by MPPI, allowing online adaptation of barrier evolution without requiring singular barriers. We further design a cost function that promotes parameter values consistent with Nagumo's condition at the safe-set boundary, thereby encouraging safety.

The resulting multiple state- and control-dependent equality constraints pose a challenge for random sampling, since sampled controls need not satisfy them simultaneously. Inspired by manifold path planning, we introduce state transformations and state-dependent control projections that map sampled controls onto the constraint manifold. For control-affine dynamics, a first-order approximation yields an analytical projection. This formulation can induce multimodal trajectory distributions in the robot's original state space, with sampled paths passing on different sides of obstacles, as illustrated in Fig. 1.

To the best of our knowledge, this is the first work to consider: 1) MPPI with multiple state- and control-dependent equality constraints; 2) set invariance theory to promote constraint satisfaction in an augmented state space accounting for barrier values and their rates of change; and 3) MPPI-based design or adaptation of CBF class- $\mathcal{K}$ function parameters. Although parameter adaptation has been studied for CBF-QPs [13]–[17], it has not been studied for MPPI.

Simulation case studies demonstrate improved sample efficiency over vanilla MPPI. In 100-trial benchmarks for each of five robot models, BR-MPPI with analytical barriers achieves higher success rates than four safety-oriented MPPI variants, with 87–100% success and zero unsafe trials. We also incorporate signed distance fields learned from robot geometry, reducing mean command computation time by factors of 2.10–2.52 relative to analytical barriers, with model-dependent effects on navigation outcomes. Quadrotor hardware experiments further demonstrate navigation in constrained environments near safe-set boundaries.

II. PRELIMINARIES

A. Notation

The set of integers, real numbers, and positive real numbers are denoted as $\mathbb{Z}$, $\mathbb{R}$, and $\mathbb{R}^+$ respectively. The time derivative of x is denoted by $\dot{x}$. Given $x \in \mathbb{R}^n$, $\|x\|_Q := \sqrt{x^T Q x}$ is called the weighted Q norm for a positive definite matrix Q . $\langle a, b \rangle = a^T b$ represents the inner product between $a, b \in \mathbb{R}^n$. The boundary of a set $\mathcal{C}$ is denoted by $\partial \mathcal{C}$. A continuous function $\alpha : [0, a) \rightarrow [0, \infty)$ is a class- $\mathcal{K}$ function if it is strictly increasing and $\alpha(0) = 0$.

Furthermore, if $a = \infty$ and $\lim_{r \rightarrow \infty} \alpha(r) = \infty$, then it is called class- $\mathcal{K}_\infty$. Both $\frac{\partial}{\partial x}$ and ∇_x denote gradient and will be used interchangeably depending on the complexity of expressions for easy understanding.

B. System Description

Consider the following discrete-time dynamics with state $x_t \in \mathbb{R}^n$, control input $u_{x_t} \in \mathbb{R}^m$

$$x_{t+1} = x_t + F(x_t, u_{x_t}) \quad (1)$$

where $F : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ is the dynamics function. The following control-affine dynamics form, a special case of (1) will also be referred to later in Section III-C.1

$$x_{t+1} = x_t + f(x_t) + g(x_t)u_{x_t} \quad (2)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ are continuous functions. Let $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^p$ project the robot state x_t to its position $\phi(x_t) \in \mathbb{R}^p$, we also denote the robot body as a set-valued function $\mathcal{B}(x_t) \subset \mathbb{R}^p$, and its surface as $\partial \mathcal{B}(x_t)$.

The state x_t is required to lie in safe sets $\mathcal{S}_i, i \in \{1, \dots, N\}$ that are defined to be 0-superlevel sets of continuously differentiable constraint functions $h_i : \mathbb{R}^n \rightarrow \mathbb{R}$

$$\mathcal{S}_i \triangleq \{x \in \mathbb{R}^n : h_i(x) \geq 0\} \quad (3a)$$

$$\partial \mathcal{S}_i \triangleq \{x \in \mathbb{R}^n : h_i(x) = 0\} \quad (3b)$$

C. Control Barrier Functions

CBFs are a common tool for enforcing safety constraints in a controller. For a linear class- $\mathcal{K}$ function $\nu_i(r) = \alpha_i r$ with $0 < \alpha_i \leq 1$, the following condition on the applied input preserves $h_i(x_t) \geq 0$:

$$h_i(x_t + F(x_t, u_{x_t})) - h_i(x_t) \geq -\alpha_i h_i(x_t). \quad (4)$$

For multiple constraints, the same input must satisfy all such inequalities simultaneously. In practice, (4) is often imposed using a constraint function h_i as a candidate CBF, even when its feasibility over the entire safe set is not guaranteed. Intuitively, the CBF condition upper bounds the rate at which the x_t can approach the safe set boundary: it allows $h(x_{t+1}) < h(x_t)$ when $h(x_t) > 0$ but requires $h(x_{t+1}) \geq h(x_t)$ when $h(x_t) = 0$. The *rate* is governed by α . Since finding valid CBFs with desired performance characteristics (e.g., aggressive or conservative) is difficult α is usually tuned offline or online.

D. Model Predictive Path Integral Control

We briefly describe the MPPI algorithm. For a time horizon H , consider the state trajectory $\mathbf{x} = [x_t^T, \dots, x_{t+H}^T]^T$, mean control input sequence $\mathbf{v} = [v_t^T, \dots, v_{t+H-1}^T]^T, v_\tau \in \mathbb{R}^m$, and Gaussian noise $\mathbf{w} = [w_t^T, \dots, w_{t+H-1}^T]^T$ where $w_\tau \sim N(0, \Sigma_w)$ and Σ_w is the noise covariance, often chosen by the user. Let the disturbed control input sequence be $\mathbf{u} = [u_t, \dots, u_{t+H-1}] = \mathbf{v} + \mathbf{w}$. MPPI then solves the

following problem

$$\min_{\mathbf{v}} \quad J(\mathbf{v}) = \mathbb{E} \left[\Phi(x_{t+H}) + \sum_{\tau=t}^{t+H-1} \left(Q(x_\tau, u_\tau) + \frac{\lambda}{2} v_\tau^\top \Sigma_w^{-1} v_\tau \right) \right] \quad (5a)$$

$$\text{s.t.} \quad x_{\tau+1} = x_\tau + F(x_\tau, v_\tau + w_\tau) \quad (5b)$$

$$w_\tau \sim \mathcal{N}(0, \Sigma_w) \quad (5c)$$

Here, $Q : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ is the stage cost, $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}$ is the terminal cost, and $\lambda > 0$ is the MPPI temperature.

MPPI uses sampling to solve (5). Given a sample size K , we sample K control input perturbation sequences $\mathbf{w}^k, k \in \{1, \dots, K\}$, where each sequence is of size H , that is $\mathbf{w}^k = [w_t^k, \dots, w_{t+H-1}^k]$. The control input sequence is then computed as $\mathbf{u}^k = \mathbf{v} + \mathbf{w}^k$. For each sequence k , we compute the states $x_\tau^k, \tau \in [t, \dots, t+H]$, using dynamics (1).

$$S_k = \Phi(x_{t+H}^k) + \sum_{\tau=t}^{t+H-1} [Q(x_\tau^k, u_\tau^k) + \gamma v_\tau^T \Sigma_w^{-1} u_\tau^k] \quad (6)$$

where $\gamma \in [0, \lambda]$. The weight w_k of the k^{th} trajectory is

$$w_k = \exp \left(-\frac{1}{\lambda} (S_k - \beta) \right) \quad (7)$$

where $\beta = \min_{k \in \{1, \dots, K\}} S_k$. The optimal control sequence is then computed as

$$\mathbf{v}^+ = \sum_{k=1}^K w_k \mathbf{u}^k / \sum_{k=1}^K w_k. \quad (8)$$

The cost functions are generally user-designed for specific objectives. We consider two such objectives: goal convergence and collision avoidance.

E. Signed Distance Function

Signed distance functions (SDFs) are widely used in robotics to accurately model environmental geometries [18]–[20]. Following recent developments [21]–[23], we employ SDFs to model the robot's shape. This approach offers greater expressiveness for complex robot geometries and enables faster distance computations in environments represented by point-clouds. In particular, for a robot body $\mathcal{B}$, we define the robot's SDF $\varphi : \mathbb{R}^p \mapsto \mathbb{R}$:

$$\varphi(\mathbf{p}) := \begin{cases} -d(\mathbf{p}, \partial\mathcal{B}), & \mathbf{p} \in \mathcal{B}, \\ d(\mathbf{p}, \partial\mathcal{B}), & \mathbf{p} \notin \mathcal{B}, \end{cases} \quad (9)$$

where d denotes the Euclidean distance between a point and a set, and $\mathbf{p} \in \mathbb{R}^p$ is a workspace point.

III. METHODOLOGY

We discuss our approach to training the robot SDF model.

A. Training the Robot SDF Model

To train the robot SDF, we prepare a dataset $\mathcal{D}$ from the robot's geometric description (e.g., URDF), containing samples $(\mathbf{p}, d)$ of workspace points $\mathbf{p} \in \mathbb{R}^p$ and their signed distances d to the robot surface $\partial\mathcal{B}$.

We train the SDF model $\hat{\varphi}(\mathbf{p}; \boldsymbol{\theta})$, parameterized by $\boldsymbol{\theta}$, using the loss

$$\ell(\boldsymbol{\theta}; \mathcal{D}) = \ell^D(\boldsymbol{\theta}; \mathcal{D}) + \lambda_E \ell^E(\boldsymbol{\theta}; \mathcal{D}), \quad (10)$$

where ℓ^D and ℓ^E are the distance and Eikonal losses, respectively, and λ_E balances them. The distance loss ℓ^D measures the mean squared error between the predicted signed distances and the ground-truth signed distances $\ell^D(\boldsymbol{\theta}; \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{p}, d) \in \mathcal{D}} (\hat{\varphi}(\mathbf{p}; \boldsymbol{\theta}) - d)^2$.

The Eikonal loss ℓ^E encourages the SDF gradient to have unit norm like a true distance function

$$\ell^E(\boldsymbol{\theta}; \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{p}, d) \in \mathcal{D}} (\|\nabla_{\mathbf{p}} \hat{\varphi}(\mathbf{p}; \boldsymbol{\theta})\| - 1)^2. \quad (11)$$

By minimizing the combined loss function in (10), we train the SDF model to accurately represent the robot's geometry.

B. Safe MPPI Formulation

In this work, instead of trying to impose the inequality constraint (4), we impose the following equality constraint

$$h_i(x_t + F(x_t, u_{x_t})) - h_i(x_t) = -\alpha_{i,t} h_i(x_t), \forall i \in \{1, \dots, N\} \quad (12)$$

and allow α_i to vary over time (hence the notation $\alpha_{i,t}$ in (12)). We achieve this by introducing a pseudo-parameter state $\tilde{\alpha}_{i,t}$ and additional control inputs $u_{\tilde{\alpha}_{i,t}}$ in the augmented system

$$\underbrace{\begin{bmatrix} x_{t+1} \\ \tilde{\alpha}_{t+1} \end{bmatrix}}_{z_{t+1}} = \underbrace{\begin{bmatrix} x_t \\ \tilde{\alpha}_t \end{bmatrix}}_{z_t} + \begin{bmatrix} F(x_t, u_{x_t}) \\ u_{\tilde{\alpha}_t} \end{bmatrix} \quad (13)$$

where $z_t = [x_t^T \tilde{\alpha}_t^T]^T \in \mathbb{R}^{n+N}$ with $\tilde{\alpha}_t = [\tilde{\alpha}_{1,t}, \dots, \tilde{\alpha}_{N,t}]^T$ and $u_t = [u_{x_t}^T u_{\tilde{\alpha}_t}^T]^T, u_{\tilde{\alpha}_t} = [u_{\tilde{\alpha}_{1,t}}, \dots, u_{\tilde{\alpha}_{N,t}}]^T$ denote the augmented state and control input. $\tilde{\alpha}_t$ will be used in Section III-C to compute the α_t for (12) at time t . Specifically, we will design parameter dynamics such that $\alpha_t = \tilde{\alpha}_{t+1}$.

We allow α_t and $\tilde{\alpha}_t$ to vary in $\mathbb{R}^N$. For $h_i(x_t) > 0$, negative $\alpha_{i,t}$ permits the barrier value to increase, so the rate function is not restricted to class- $\mathcal{K}$. Section III-D introduces a cost function to promote constraint satisfaction.

Remark 1. We introduce $\tilde{\alpha}_t$ rather than α_t , as the new state variable because α_t and u_t must satisfy (12) for $i \in \{1, \dots, N\}$ simultaneously. This compatibility cannot be enforced if α_t were a state variable that is guided by random MPPI sampled control inputs. Similarly, treating α_t as a control input would constrain the joint choice of u_t and α_t , precluding MPPI's random Gaussian sampling. We detail our procedure for ensuring compatibility below.

Remark 2. The notion of designing dynamics for parameter α is similar to existing works that design $\hat{\alpha}$ in discrete-time [13] and continuous-time [14], [15].

Before presenting our final algorithm, we introduce another control space transformation. Let $u'_{x_t} \in \mathbb{R}^m$ and $u'_{\tilde{\alpha}_t} \in \mathbb{R}^N$ be pseudo-inputs that are randomly sampled by MPPI and u_{x_t} and $u_{\tilde{\alpha}_t}$ are obtained from pseudo-inputs through a projection operation $\mathbb{P} : \mathbb{R}^{n+N} \times \mathbb{R}^m \times \mathbb{R}^N \rightarrow \mathbb{R}^m \times \mathbb{R}^N$ introduced in the next section. MPPI therefore considers the dynamics

$$\begin{bmatrix} x_{t+1} \\ \tilde{\alpha}_{t+1} \end{bmatrix} = \begin{bmatrix} x_t \\ \tilde{\alpha}_t \end{bmatrix} + \begin{bmatrix} F(x_t, u_{x_t}) \\ u_{\tilde{\alpha}_t} \end{bmatrix}, \quad (14a)$$

$$(u_{x_t}, u_{\tilde{\alpha}_t}) = \mathbb{P}(z_t, u'_{x_t}, u'_{\tilde{\alpha}_t}). \quad (14b)$$

The projection $\mathbb{P}$ maps randomly sampled inputs onto the manifolds defined by (12).

C. Design of the Projection Function

The system of equations (12) might not admit a solution for arbitrary values of u_{x_t} and α_t . Therefore, we design $\mathbb{P}$ to ensure their compatibility for all $i \in \{1, \dots, N\}$.

Let u'_{x_t} and $u'_{\tilde{\alpha}_t}$ be randomly sampled control inputs from MPPI. We project them onto the constraint manifolds as

$$\mathbb{P}(z_t, u'_{x_t}, u'_{\tilde{\alpha}_t}) = \arg \min_{v, a} \|v - u'_{x_t}\|_{Q_1}^2 + \|a - u'_{\tilde{\alpha}_t}\|_{Q_2}^2 \quad (15a)$$

$$\text{s.t. } h_i(x_t + F(x_t, v)) - h_i(x_t) = -(\tilde{\alpha}_{i,t} + a_i)h_i(x_t), \quad i = 1, \dots, N. \quad (15b)$$

Since $\tilde{\alpha}_{i,t} + a_i = \tilde{\alpha}_{i,t+1}$ the dynamics in (14) ensure $\tilde{\alpha}_{t+1} = \alpha_t$.

1) *Simplification for Control Affine Dynamics:* The optimization (15) can be expensive for general dynamics F . We therefore restrict our discussion to the special case of control-affine dynamics in (2), for which a first-order approximation of (15) admits an analytical solution. An extension to general dynamics is left for future work. Under (2), linearizing h about x_t gives

$$\begin{aligned} h(x_t + f(x_t) + g(x_t)u_{x_t}) - h(x_t) \\ \approx \left. \frac{\partial h}{\partial x} \right|_{x_t} (f(x_t) + g(x_t)u_{x_t}). \end{aligned} \quad (16)$$

making the constraints in (15) linear in u_t . The projection thus reduces to an equality-constrained weighted minimum norm problem:

$$\begin{aligned} \mathbb{P}(z_t, u'_{x_t}, u'_{\tilde{\alpha}_t}) = \arg \min_z (z - z_{\text{des}})^T W (z - z_{\text{des}}) \\ \text{s.t. } A_t z = b_t. \end{aligned} \quad (17)$$

where

$$\begin{aligned} z = \begin{bmatrix} v \\ a \end{bmatrix}, \quad z_{\text{des}} = \begin{bmatrix} u'_{x_t} \\ u'_{\tilde{\alpha}_t} \end{bmatrix}, \quad W = \text{diag}(Q_1, Q_2), \\ (A_t)_{i,:} = [\nabla h_i(x_t)^T g(x_t) \quad h_i(x_t)e_i^T], \\ (b_t)_i = -\nabla h_i(x_t)^T f(x_t) - \tilde{\alpha}_{i,t}h_i(x_t). \end{aligned}$$

Here, e_i is the i -th standard basis vector of $\mathbb{R}^N$, and $Q_1 \succ 0$ and $Q_2 \succ 0$ are weighting matrices.

The solution to (17) is

$$z = z_{\text{des}} + W^{-1}A^T(AW^{-1}A^T)^{-1}(b - Az_{\text{des}}) \quad (18)$$

Algorithm 1 BR-MPPI Controller

Require: z_t, H, F, Q_c $\triangleright$ current augmented state, time horizon, dynamics function, user-given convergence cost

- 1: Sample $w_\tau^k \sim \mathcal{N}(0, \Sigma_w)$, $k = 1, \dots, K$, $\tau = t, \dots, t + H - 1$
- 2: $\mathbf{u}^k \leftarrow \mathbf{v} + \mathbf{w}^k$
- 3: **for** $k = 1$ to K **do** $\triangleright$ Loop over samples; parallel in GPU
- 4: $z_t^k \leftarrow z_t$, $S_k \leftarrow 0$ $\triangleright$ Initialize Cost
- 5: **for** $\tau = t$ to $t + H - 1$ **do** $\triangleright$ Loop over time horizon
- 6: $u'_{x_\tau}, u'_{\tilde{\alpha}_\tau} \leftarrow \mathbf{u}_\tau^k$
- 7: Compute $u_{x_\tau}, u_{\tilde{\alpha}_\tau}$ using $\mathbb{P}$ in (15) or (18)
- 8: Compute next state $z_{\tau+1}$ using (14)
- 9: Compute Q_h using (20)
- 10: $S_k \leftarrow S_k + Q_c(x_\tau^k) + Q_h + \gamma(v_\tau)^T \Sigma_w^{-1}(u_\tau^k)$
- 11: **end for**
- 12: $S_k \leftarrow S_k + \Phi(x_{t+H}^k)$
- 13: **end for**
- 14: $w_k \leftarrow (7)$ $\triangleright$ Compute weights
- 15: $\mathbf{v}^+ \leftarrow (8)$ $\triangleright$ Computed final control trajectory
- 16: **return** $\mathbf{v}^+$

Remark 3. Sampling inputs subject to equality constraints is actively studied for path planning on manifolds [24], e.g., for closed-chain systems with state constraints. Our projection is inspired by such works, as each equality constraint in (12) induces a manifold for the state-action pair. Consequently, our method inherits discretization errors associated with manifold constraints. While (17) performs well in practice, its first-order approximation can introduce errors for highly nonlinear h , potentially causing instability. One could recompute the tangent space of h_i when this error exceeds a threshold, as in [25] for kinodynamic RRT; we leave its application to real-time control for future work. Note that the dynamics (1) itself defines a manifold inherently followed by MPPI trajectories. However, to the best of our knowledge, this is the first work considering multiple state- and input-dependent equality constraints in MPPI.

D. Design of MPPI Cost

We decompose the MPPI stage cost Q into Q_c and Q_h , which promote goal convergence and constraint satisfaction, respectively. Q_c is user-designed; below, we define Q_h .

A necessary condition for constraint satisfaction is given by Nagumo's theorem [26], [27] and states that whenever $h_i(x_t) = 0$, $\dot{h}_i(x_t) \geq 0$, or in discrete time, $h(x_{t+1}) - h(x_t) = -\alpha_t h(x_t) \geq 0$. To account for discretization errors, we consider a buffer zone D_i of length d_i near the constraint boundary:

$$D_i = \{x \mid 0 \leq h_i(x) \leq d_i\} \quad (19)$$

Inside D_i , we promote $\alpha_{i,t} < 0$ in (12) to increase h_i . The buffer length d_i is a tuning parameter. Since $\alpha_{i,t} \in \mathbb{R}$, the robot can explore different motions around obstacles while Nagumo's condition is promoted only near the boundary:

$$Q_h(z_t, z_{t+1}, u) = \sum_{i=1}^N \mathbf{1}(x_t \in D_i \cup x_t \in \bar{\mathcal{S}}_i) \frac{\tilde{\alpha}_{i,t+1}}{h_i(x)} \quad (20)$$

The complete algorithm is summarized in Algorithm 1.

E. Discussion

While applying MPPI to the augmented system (13) can be viewed as a state transformation, it has several important

consequences. Here, we discuss its differences and improvements over standard MPPI and CBF approaches.

1) *Improvement of MPPI Through CBF*: First, we use the well-established theory of CBFs and set invariance to guide inequality constraint satisfaction. Our algorithm handles inequality constraints through two steps: 1) replacing each inequality constraint with a set of equality constraints parameterized by the additional state $\tilde{\alpha}$, and 2) using manifold planning techniques to satisfy these equality constraints. Their satisfaction is promoted through Nagumo's theorem near the constraint boundary.

Second, Fig. 1 shows sampled trajectories passing on different sides of obstacles. The state-dependent projection transforms Gaussian pseudo-input samples before rollout, changing the resulting trajectory distribution. Our method can still sample trajectories inside obstacles, but avoids such solutions more effectively in the reported examples.

2) *Improvement of CBF Through MPPI*: Finding valid CBFs remains an active research area, and candidate CBFs are commonly used in CBF-QP controllers to impose (4). A candidate CBF may yield infeasible CBF-QPs for some states, while even a valid CBF may be overly conservative for a given class- $\mathcal{K}$ function. Several works therefore adapt the class- $\mathcal{K}$ function offline or online. Our algorithm can be viewed as adapting the class- $\mathcal{K}$ function online to improve performance while promoting safety. Moreover, although we use CBF-like conditions, constraint satisfaction is enforced through Nagumo's theorem only near the boundary, while $\tilde{\alpha}$ can vary in $\mathbb{R}$ in the safe-set interior. Our method is therefore less restrictive than a fixed-class- $\mathcal{K}$ CBF. Further analysis of the relationship between our MPPI solutions and valid CBFs is left for future work.

IV. SIMULATION RESULTS

Videos and code for all simulations and hardware experiments are accessible from our project page and paper attachments.

We use Algorithm 1 to navigate the robot in obstacle environments. We consider the following five dynamics models: single integrator (SI), unicycle (U), dynamic unicycle (DU), mobile arm (MA), and a planar quadrotor (PQ).

$$\text{SI: } \dot{p}_x = u_1, \dot{p}_y = u_2 \quad (21)$$

$$\text{U: } \dot{p}_x = v \cos \theta, \dot{p}_y = v \sin \theta, \dot{\theta} = u_1 \quad (22)$$

$$\text{DU: } \dot{p}_x = v \cos \theta, \dot{p}_y = v \sin \theta, \dot{\theta} = u_1, \dot{v} = u_2 \quad (23)$$

$$\text{MA: } \dot{p}_x = u_1, \dot{p}_y = u_2, \dot{\theta}_{j,i} = u_{j,i}, j \in \{l, r\}, i \in \{1, 2, 3\} \quad (24)$$

$$\text{PQ: } \ddot{p}_x = u_1, \ddot{p}_y = u_2, \ddot{\theta} = u_3 \quad (25)$$

where $p_x, p_y \in \mathbb{R}$ denote the planar position of the robot base, $\theta \in \mathbb{R}$ its heading, and $v \in \mathbb{R}$ its forward velocity. For the mobile arm, $\theta_{j,i}$ denotes the i th joint angle of arm j , where $j \in \{l, r\}$ identifies the left or right arm and $i \in \{1, 2, 3\}$. The variables u_1 and u_2 denote model-specific control inputs, corresponding to translational velocities, acceleration, angular velocity and acceleration as specified by the respective dynamics. For non-point robot footprints, we

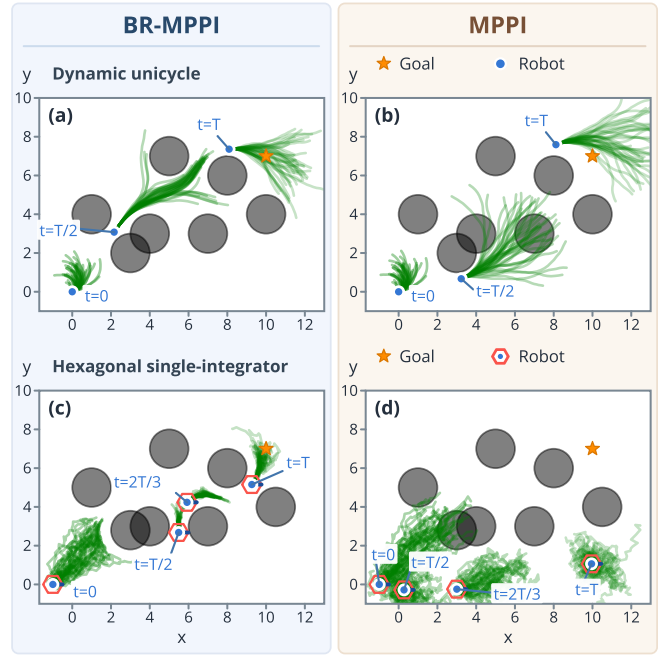


Fig. 2. (a),(b) Dynamic unicycle and (c), (d) Hexagonal single-integrator robots navigating an obstacle environment with BR-MPPI and MPPI. T denotes the minimum of two values: the time required to reach within 2 meters of the goal (which varies across simulations) and the total simulation duration.

use the signed distance representation described in Section III-A.

A. Comparative Study - Feasibility over Vanilla MPPI

We compare the proposed algorithm with vanilla MPPI to evaluate its ability to navigate robots with different shapes and dynamics without modifying the MPPI parameters, i.e., without fine-tuning MPPI for each new robot shape or dynamics model. We simulate both methods with $K = 1000$, $\gamma = 0.1$ and $\lambda = 0.03$ in (7) but also normalize the costs to $[0, 1]$. $Q_c(x) = \min(\|\phi(x) - p_d\|^2, 100)$, where $p_d \in \mathbb{R}^p$ is the goal position and Q_h is as defined in (20) for BR-MPPI and $Q_h = 1/\min_i(h_i)$ for MPPI. For second-order systems, we use the velocity-dependent candidate $h_i = \delta_i + 5\Delta t \dot{\delta}_i$, where δ_i denotes signed obstacle clearance and Δt is the simulation time step. Fig. 2(a), (b) shows snapshots of the paths generated by vanilla and proposed MPPI for a point-mass DU robot. The proposed algorithm successfully passes through narrow spaces, whereas vanilla MPPI takes a detour. Increasing the number of samples for vanilla MPPI to 20,000 results in paths similar to those obtained by our algorithm, demonstrating better sample efficiency of the proposed method in this case study. Next, we apply both algorithms to a hexagonal SI robot and visualize the resulting paths in Fig. 2(c), (d). Vanilla MPPI is overly conservative, causing the robot to remain near its initial position, whereas our algorithm successfully finds a path to the goal. Simulation videos for different robot dynamics are available on our website and the attachment.

Representative trials

Trajectory + robot footprints, with matched clearance below

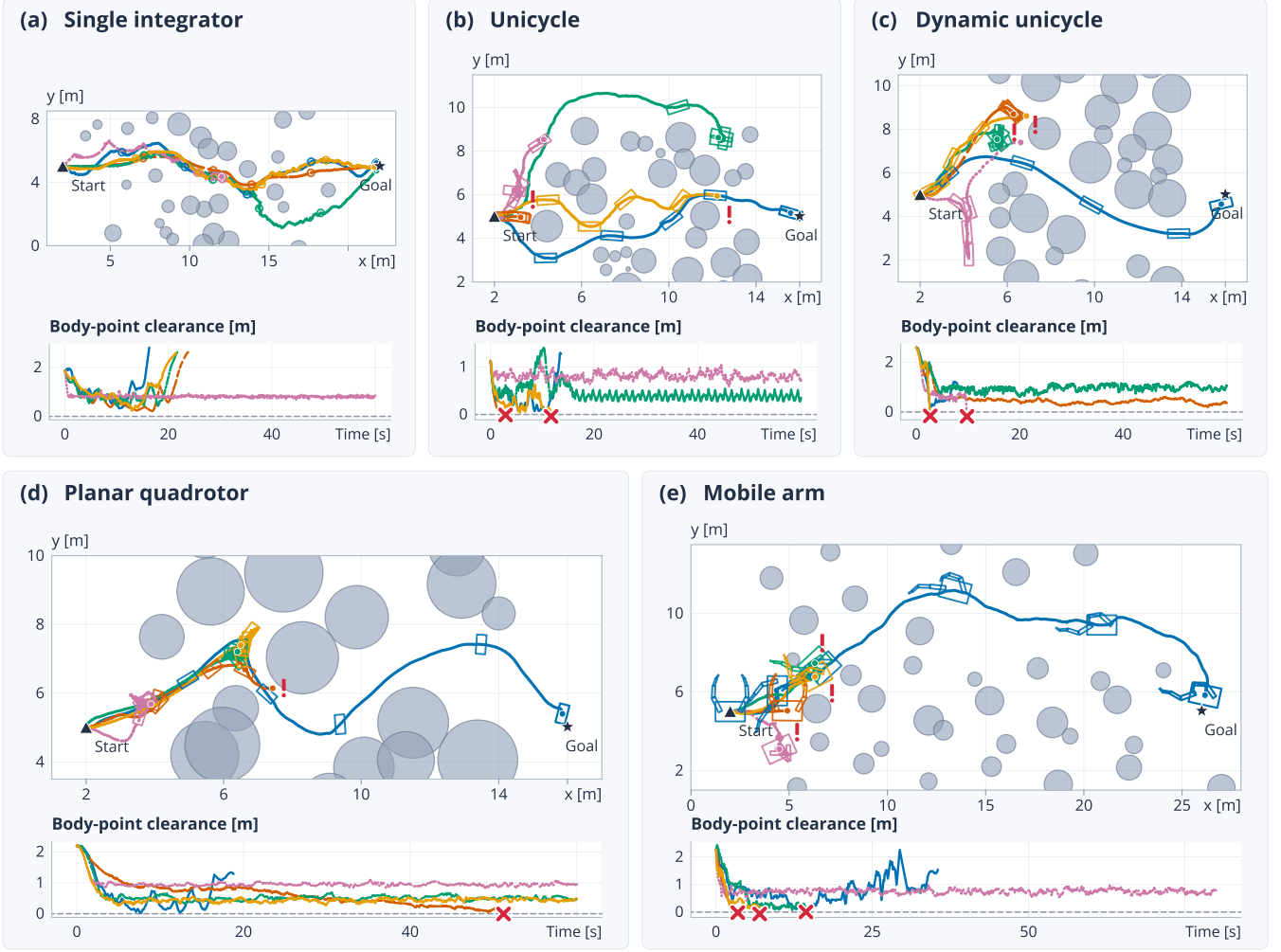


Fig. 3. Representative trials comparing BR-MPPI with MPPI-CBF, Shield-MPPI, SC-MPPI, and GS-MPPI across five robot models: (a) single integrator, (b) unicycle, (c) dynamic unicycle, (d) planar quadrotor, and (e) mobile arm. Each panel shows trajectories with robot footprints and the corresponding body-point clearance histories. Red exclamation marks in the trajectory plots and red crosses in the clearance plots indicate collisions.

TABLE I

ANALYTIC VERSUS OPTIMIZED NEURAL SDF BARRIERS IN BR-MPPI OVER 100 TRIALS PER ROBOT MODEL. R/C/T DENOTES REACHED/COLLISION/TIMEOUT; DEADLOCKS ARE INCLUDED IN TIMEOUTS. SPEEDUP IS THE RATIO OF ANALYTIC TO NEURAL LATENCY.

Robot model	Analytic SDF			Optimized neural SDF			
	R / C / T	Success	Avg. Comp. Time (ms)	R / C / T	Success	Avg. Comp. Time (ms)	Speedup
Unicycle	87 / 0 / 13	87%	38.82	84 / 11 / 5	84%	18.46	2.10 ×
Dynamic unicycle	100 / 0 / 0	100%	47.76	100 / 0 / 0	100%	18.97	2.52 ×
Planar quadrotor	98 / 0 / 2	98%	52.23	94 / 0 / 6	94%	23.48	2.22 ×

B. Comparative Study - MPPI and CBF variants

We compare BR-MPPI against four safety-oriented MPPI baselines: MPPI-CBF, which filters the MPPI command through a CBF-QP; Shield-MPPI [8], which applies CBF-based shielding to the MPPI solution; SC-MPPI [11], which incorporates barrier-state-based safe feedback into sampling,

while GS-MPPI [10] applies a closed-form CBF-based safety correction. We consider five dynamics—single integrator, unicycle, dynamic unicycle, planar quadrotor, and mobile arm—and tune all methods independently for each case study. Within each case study, the shared MPPI parameters, including the number of samples K and horizon H , are fixed

Benchmark results

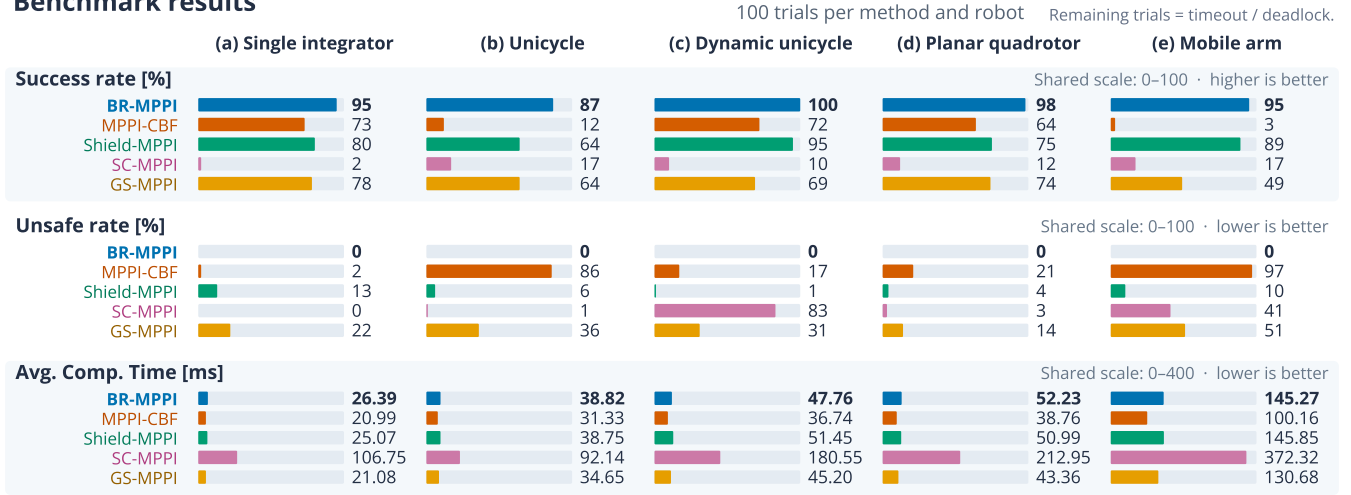


Fig. 4. Benchmark results over 100 trials per method and robot model. Columns (a)–(e) follow the robot-model order in Fig. 3. Rows show success rate, unsafe rate, and mean command computation time. Trials that are neither successful nor unsafe terminate in timeout or deadlock.

across methods, while method-specific hyperparameters are optimized independently using Optuna, with configurations selected primarily by task success.

All methods share quadratic penalties on goal distance and control effort, together with a terminal goal-distance penalty. MPPI-CBF filters the selected action using a CBF-QP, while Shield-MPPI penalizes barrier violations and repairs the control sequence. SC-MPPI incorporates inverse-barrier states into its feedback optimization, whereas GS-MPPI applies a closed-form safety correction. BR-MPPI, MPPI-CBF, and Shield-MPPI retain separate obstacle barriers. SC-MPPI combines them using a minimum when configured to do so, while GS-MPPI uses a smooth minimum. BR-MPPI additionally adapts a separate barrier-rate variable for each obstacle.

Figure 3 shows representative trajectories and the corresponding body-point clearance histories for the five methods across the five robot models. BR-MPPI reaches the goal without collision in all five representative trials. In particular, panels (c)–(e) illustrate cases where the competing methods collide or fail to reach the goal.

In the 100-trial benchmarks shown in Fig. 4, BR-MPPI achieves the highest success rate for every robot model (87–100%) and is the only method with zero unsafe trials across all five models. The remaining BR-MPPI trials terminate in timeout or deadlock. Its mean command computation time ranges from 26.39 to 145.27 ms, exceeding those of MPPI-CBF and GS-MPPI but remaining below that of SC-MPPI for every model.

These violations do not contradict the baselines’ conditional safety guarantees. Distance-based candidate CBFs may be infeasible under bounded inputs; SC-MPPI relies on barrier-state feedback that is locally valid around the nominal trajectory; and GS-MPPI can lose its CBF guarantee when actuator saturation modifies the closed-form safe control.

C. Comparative Study: Exact vs Learned SDF

We compare analytic and learned signed-distance barriers within BR-MPPI over 100 trials for each of the three robot models in Table I. We retain the tuned BR-MPPI controller parameters from Section IV-B and change only the barrier-function implementation from the analytical distance representation to its learned counterpart. The results are shown in Table I. The optimized neural SDF reduces mean command latency by factors of 2.10–2.52, with model-dependent effects on navigation outcomes. For the dynamic unicycle, both formulations achieve 100 successful trials without collisions. For the planar quadrotor, the neural formulation remains collision-free, while success decreases from 98% to 94%. For the unicycle, success decreases from 87% to 84%, and 11 trials terminate in collision. While safety depends on the accuracy of the learned geometry, these results demonstrate the computational benefit of learned barriers in achieving real-time execution speeds.

V. HARDWARE EXPERIMENTS

We validate the proposed algorithm through hardware experiments on a quadrotor navigating an environment with obstacles. The quadrotor is fitted with metal straws to form a rectangular footprint of size 0.28×0.56 m and follows the planar quadrotor dynamics in (25), with positions p_x, p_y and heading angle θ . The system is implemented in ROS 2, with localization provided by a Vicon motion-capture system. The experimental setup is shown in Fig. 5(a), where the objective is to reach the goal while avoiding obstacles.

Each obstacle is represented by a point cloud of 30 points. Fig. 5(b) shows the corresponding visualization of a hardware run with BR-MPPI. In the same environment, vanilla MPPI is unable to enter the corridor, with its trajectories omitted due to space constraints. We use the same parameters as in Section IV-A and observe that, in this highly constrained environment, vanilla MPPI fails to complete the

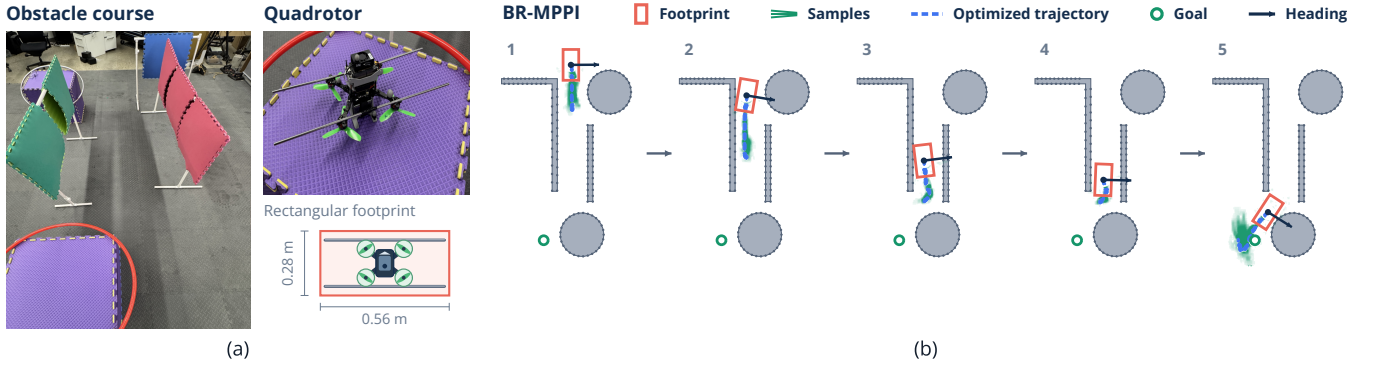


Fig. 5. (a) Experimental setup showing the obstacle course and our custom-assembled quadrotor. (b) Visualization of a hardware run with BR-MPPI. Video available on our project page.

task, whereas BR-MPPI successfully reaches the goal. A video of the hardware experiment is available on our project page and in the supplementary video.

By augmenting the state with $\tilde{\alpha}$ and optimizing its discrete-time increments, we influence how the robot approaches or moves away from obstacles. This capability is particularly important in constrained environments, where feasible trajectories may require operating close to constraint boundaries. A rigorous theoretical characterization of how this augmented-state reformulation alters the behavior of vanilla MPPI remains an important direction for future work.

VI. CONCLUSION AND FUTURE WORK

We presented BR-MPPI, which guides control sampling through CBF-like *equality constraints* by augmenting the state with barrier-rate parameters and projecting sampled controls. Simulations demonstrate improved sample efficiency over vanilla MPPI and higher success rates than safety-oriented MPPI variants, while quadrotor experiments demonstrate navigation near safe-set boundaries. Learned signed distance fields reduce computation time, with model-dependent effects on navigation outcomes. Future work includes incorporating higher-order CBFs, establishing theoretical guarantees for constraint satisfaction under actuation constraints, and extending hardware evaluations to a broader range of robotic systems.

REFERENCES

- [1] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018.
- [2] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2016.
- [3] C. Tao, H. Kim, H. Yoon, N. Hovakimyan, and P. Voulgaris, "Control barrier function augmentation in sampling-based control algorithm for sample efficiency," in *American Control Conference (ACC)*, 2022, pp. 3488–3493.
- [4] M. S. Gandhi, B. Vlahov, J. Gibson, G. Williams, and E. A. Theodorou, "Robust model predictive path integral control: Analysis and performance guarantees," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1423–1430, 2021.
- [5] J. Yin, Z. Zhang, E. Theodorou, and P. Tsotras, "Trajectory distribution control for model predictive path integral control using covariance steering," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 1478–1484.
- [6] E. Trevisan and J. Alonso-Mora, "Biased-mpci: Informing sampling-based model predictive control by fusing ancillary controllers," *IEEE Robotics and Automation Letters*, 2024.
- [7] J. Carius, R. Ranftl, F. Farshidian, and M. Hutter, "Constrained stochastic optimal control with learned importance sampling: A path integral approach," *The International Journal of Robotics Research*, vol. 41, no. 2, pp. 189–209, 2022.
- [8] J. Yin, C. Dawson, C. Fan, and P. Tsotras, "Shield model predictive path integral: A computationally efficient robust mpc method using control barrier functions," *IEEE Robotics and Automation Letters*, 2023.
- [9] P. Tsotras, K. Berntorp, *et al.*, "Chance-constrained information-theoretic stochastic model predictive control with safety shielding," *arXiv preprint arXiv:2408.00494*, 2024.
- [10] P. Rabiee and J. B. Hoagg, "Guaranteed-safe mppi through composite control barrier functions for efficient sampling in multi-constrained robotic systems," in *IEEE Conference on Decision and Control (CDC)*, 2025, pp. 5515–5520.
- [11] M. Gandhi, H. Almubarak, and E. Theodorou, "Safe importance sampling in model predictive path integral control," *arXiv preprint arXiv:2303.03441*, 2023.
- [12] M. Gandhi, H. Almubarak, Y. Aoyama, and E. Theodorou, "Safety in augmented importance sampling: Performance bounds for robust mppi," *arXiv preprint arXiv:2204.05963*, 2022.
- [13] H. Parwana and D. Panagou, "Recursive feasibility guided optimal parameter adaptation of differential convex optimization policies for safety-critical systems," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 6807–6813.
- [14] H. Parwana, A. Mustafa, and D. Panagou, "Trust-based rate-tunable control barrier functions for non-cooperative multi-agent systems," in *IEEE Conference on Decision and Control (CDC)*, 2022, pp. 2222–2229.
- [15] W. Xiao, C. Belta, and C. G. Cassandras, "Adaptive control barrier functions," *IEEE Transactions on Automatic Control*, vol. 67, no. 5, pp. 2267–2281, 2021.
- [16] J. Zeng, B. Zhang, Z. Li, and K. Sreenath, "Safety-critical control using optimal-decay control barrier function with guaranteed pointwise feasibility," in *American Control Conference (ACC)*, 2021, pp. 3856–3863.
- [17] H. Ma, B. Zhang, M. Tomizuka, and K. Sreenath, "Learning differentiable safety-critical control using control barrier functions for generalization to novel environments," in *European Control Conference (ECC)*, 2022, pp. 1301–1308.
- [18] L. Han, F. Gao, B. Zhou, and S. Shen, "Fiesta: Fast incremental Euclidean distance fields for online motion planning of aerial robots," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.
- [19] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3d Euclidean signed distance fields for on-board mav planning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1366–1373.

- [20] K. Long, C. Qian, J. Cortés, and N. Atanasov, "Learning barrier functions with memory for robust safe navigation," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4931–4938, 2021.
- [21] K. Long, Y. Yi, Z. Dai, S. Herbert, J. Cortés, and N. Atanasov, "Sensor-based distributionally robust control for safe robot navigation in dynamic environments," *arXiv preprint arXiv:2405.18251*, 2024.
- [22] Y. Li, Y. Zhang, A. Razmjoo, and S. Calinon, "Representing robot geometry as distance fields: Applications to whole-body manipulation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 15 351–15 357.
- [23] K. Long, H. Parwana, G. Fainekos, B. Hoxha, H. Okamoto, and N. Atanasov, "Neural configuration distance function for continuum robot control," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 358–364.
- [24] Z. Kingston, M. Moll, and L. E. Kavraki, "Sampling-based methods for motion planning with constraints," *Annual review of control, robotics, and autonomous systems*, vol. 1, no. 1, pp. 159–185, 2018.
- [25] R. Bordalba, L. Ros, and J. M. Porta, "A randomized kinodynamic planner for closed-chain robotic systems," *IEEE Transactions on Robotics*, vol. 37, no. 1, pp. 99–115, 2020.
- [26] M. Nagumo, "Über die lage der integralkurven gewöhnlicher differentialgleichungen," *Proceedings of the physico-mathematical society of Japan. 3rd Series*, vol. 24, pp. 551–559, 1942.
- [27] F. Blanchini, "Set invariance in control," *Automatica*, vol. 35, no. 11, pp. 1747–1767, 1999.